

1.- INTRODUCCIÓN A GIT EN VISUAL STUDIO

El objeto de este documento es introducir el uso de git en Visual Studio Code.

Para una consulta más detallada hacer el curso de Git de OpenWebinars.

Lo primero es instalar Git. Basta con descargar el paquete apropiado y proceder a su ejecución. Una vez instalado hay que asegurarse que es accesible desde línea de comandos el comando git.exe (añadir a la variable de sistema PATH la ruta donde se encuentra este comando, C:\Program Files\Git\bin)

2.- CONCEPTOS BASICOS

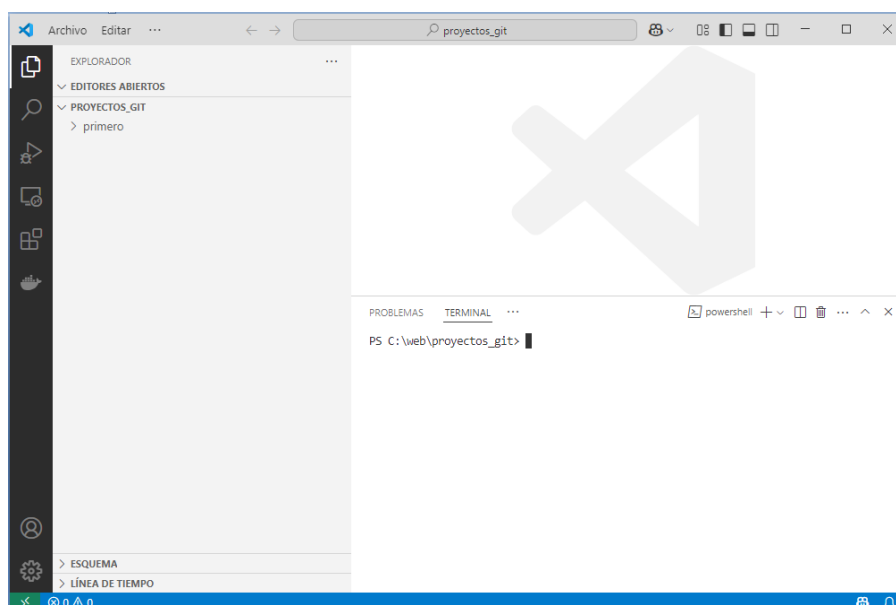
Git es un sistema de control de versiones, donde se guardan instantáneas de los archivos de nuestro proyecto. Con estas versiones podemos hacer operaciones como recuperar un archivo a una versión concreta o comparar archivos entre diferentes versiones. Siempre se tiene una versión que es con la que estamos trabajando. A esta versión se le llama HEAD.

Git trabaja con el concepto de ramas. Cada rama es un flujo de trabajo de forma que tendré mi flujo principal y tantos flujos como necesite y que combinaré una vez finalizado el desarrollo.

Además, Git define dos áreas donde se van ubicando los archivos modificados: área de trabajo y área staged. En el área de trabajo aparecen los archivos que se han modificado/añadido, es decir, todos los archivos se han cambiado desde su última versión confirmada. En el área staged, aparecen aquellos archivos que vamos a confirmar. Podemos tener el mismo archivo, con distinto contenido, en las dos áreas.

Git se puede gestionar mediante comandos o de forma gráfica. Es importante, saber los comandos ya que Git puede usarse en diferentes entornos y sistemas operativos, muchos de ellos no gráficos. En este manual haré un recorrido por las operaciones más comunes tanto en su formato gráfico como de línea de comandos.

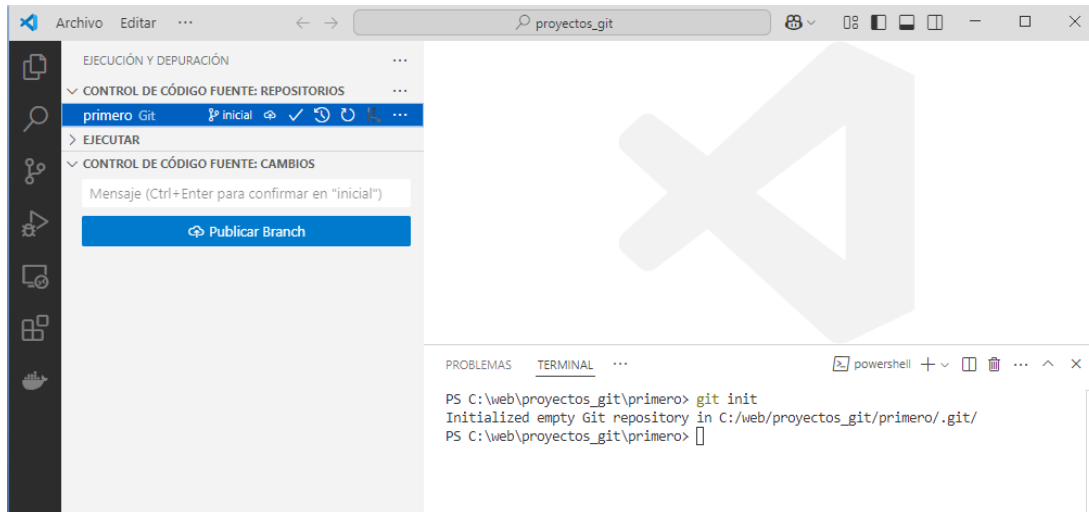
Para indicar los comandos utilizaré el terminal, bien el que trae por defecto o el terminal git bash que nos da información adicional



3.- OPERACIONES BASICAS

3.1.- Iniciar un repositorio GIT.

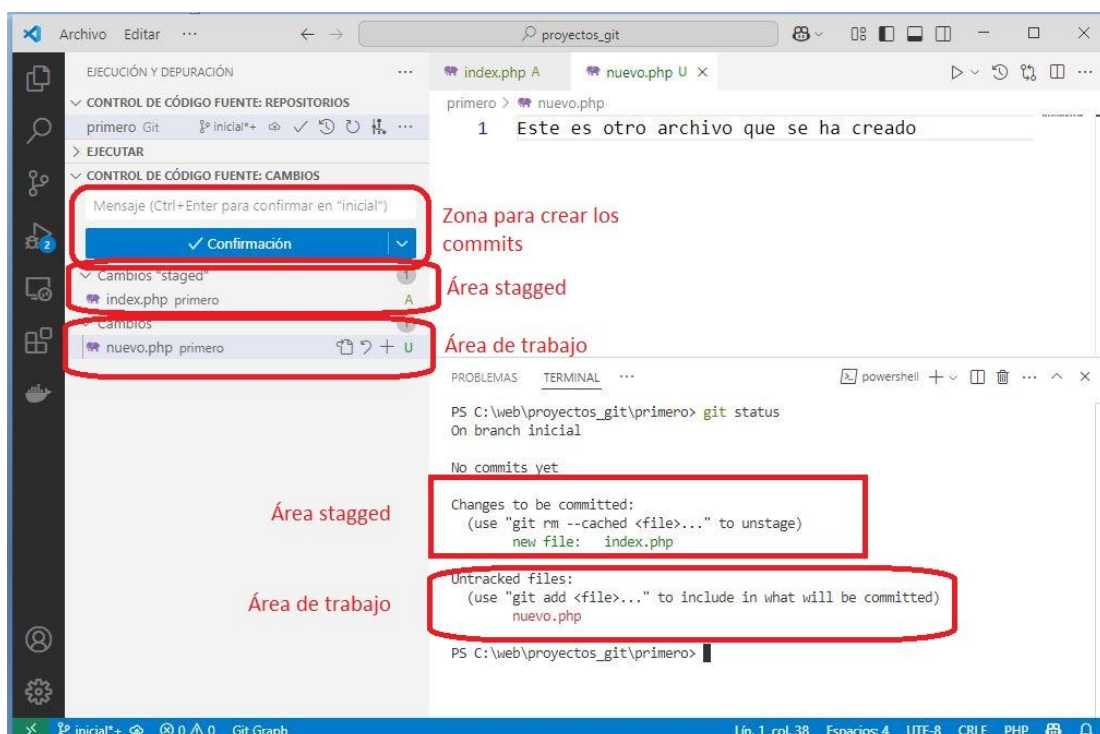
Estando en la carpeta para la que quiero llevar el control de versiones, ejecuto el comando: **git init**



Este comando realiza la inicialización del repositorio en la carpeta correspondiente. Para acceder al repositorio de forma gráfica en Visual Studio Code, nos tenemos que ir a la opción  Control de código fuente.

3.2.- Trabajar con las áreas de git

Para saber que archivos están en el área de trabajo/área staged ejecutaría el comando **git status**. Además, podemos ver esta misma información de forma gráfica en Ejecución y depuración.



Para pasar archivos del área de trabajo al área staged usamos el comando **git add archivo**. También podemos de forma gráfica en el botón + del archivo en el área de trabajo.

Para pasar archivos del área staged al área de trabajo tenemos el comando **git restore –staged archivo** o directamente en el – del archivo en el área staged.

Para volver a la versión confirmada de un archivo desde el área de trabajo se usa el comando **git restore archivo** o gráficamente con el botón – del archivo en el área de trabajo. **Ojo**, esta opción te retorna el archivo al último estado confirmado, perdiéndose por tanto todas las modificaciones que se le hayan hecho.

3.3.- Crear una versión (commit).

La confirmación se hace con el comando **git commit –m “mensaje”**

De forma gráfica indicaremos el mensaje en la zona de commits y pulsaremos el botón correspondiente.

Un commit versiona todos los archivos que tengamos en la zona staged.

Podemos ver que commits tenemos mediante el comando **git log** o en la zona gráfica de Control de Código Fuente.



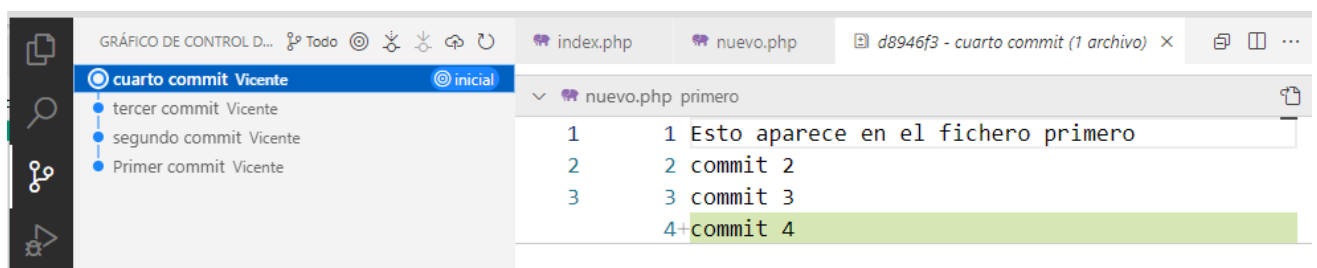
3.4.- Volver a una versión anterior (un commit anterior)

Se puede volver a versiones de ficheros anteriores. Recuerden que la última versión es HEAD y que podemos indicar una anterior mediante HEAD~2 (HEAD – 2).

Existen tres modos de hacer esto: soft (ponen los archivos en el área stagge), mix (pone los archivos en el área de trabajo) y hard (vuelve a una versión borrando las posteriores, muy peligrosa)

El comando sería **git reset --modo HEAD~numero**.

Partamos de cuatro versiones en las que en el archivo nuevo.php se han ido poniendo nuevas líneas.

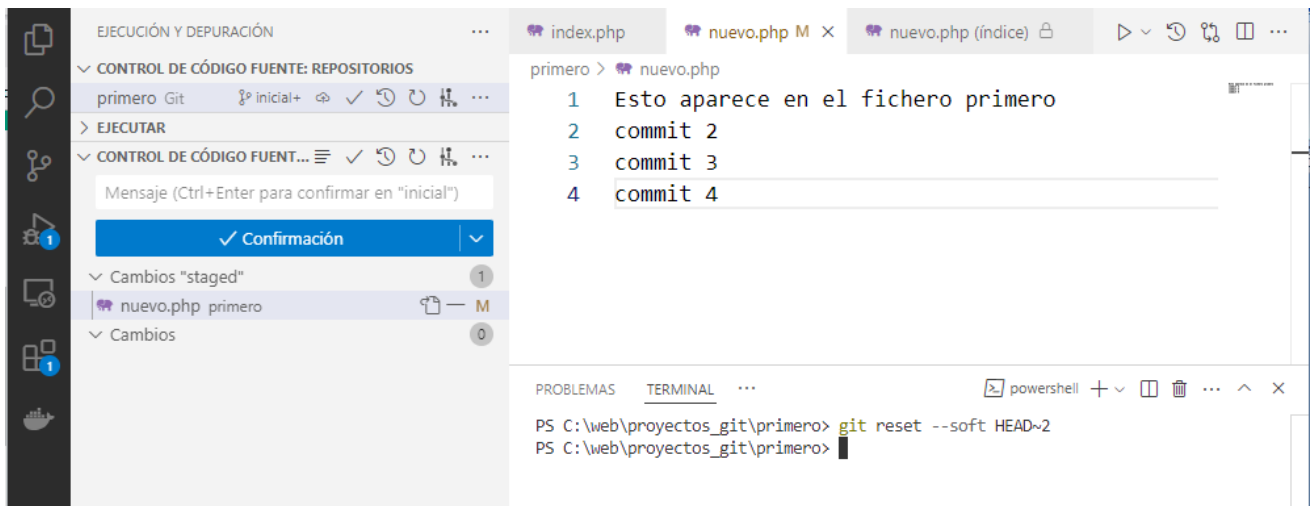


Tenemos cuatro versiones (commits), siendo el HEAD “cuarto commit”, HEAD~1 “tercer commit” y así sucesivamente

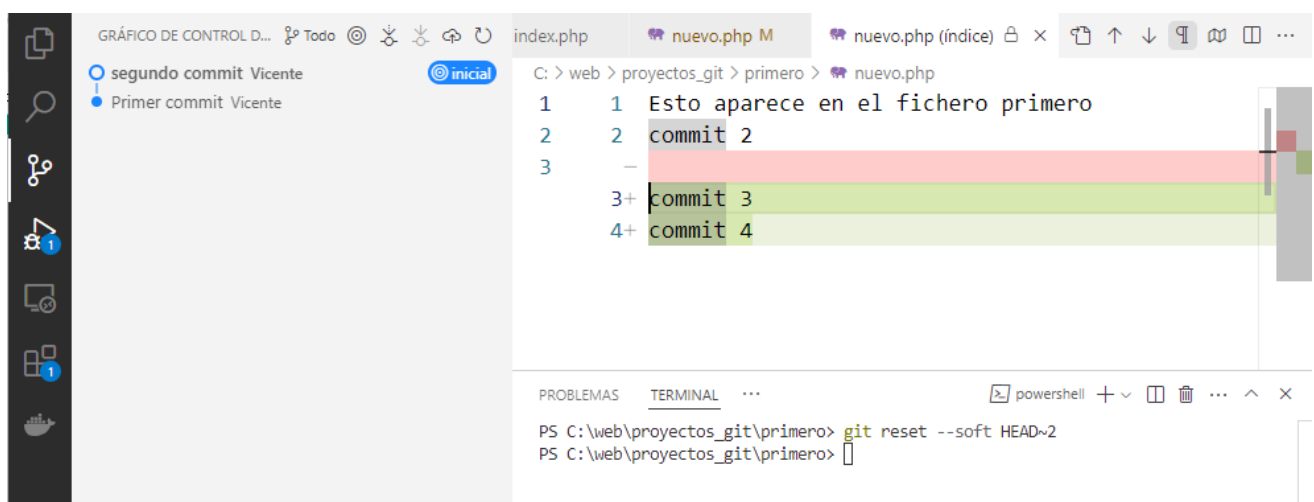
Sobre esto vamos a hacer diferentes reset:

- **git reset –soft HEAD~2** Nos pondría en el área staged los archivos correspondientes a la versión HEAD-2

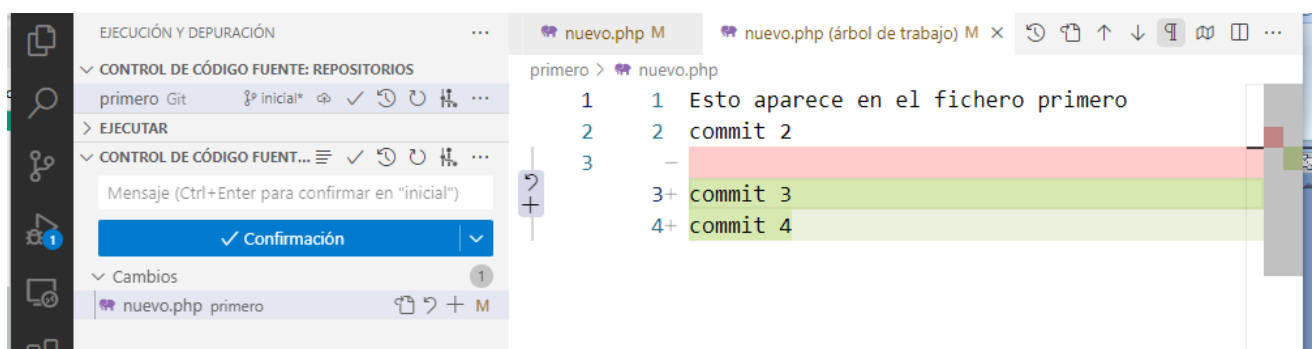
El área staged y el área de trabajo quedaría como indica la siguiente captura. El archivo nuevo.php no se ha modificado



Si vemos las versiones, han desaparecido la tercera y cuarta. Además, si abrimos el archivo nuevo.php en el área staged se vería como se observa: lo que teníamos en las versiones posteriores

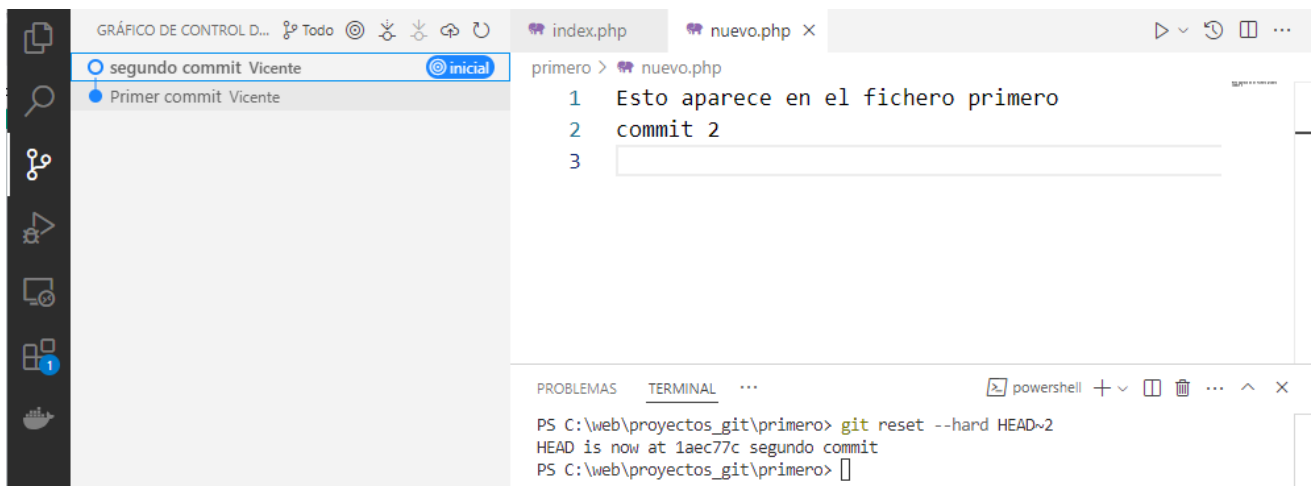


- **git reset --mix HEAD~2** Nos pondría en el área de trabajo los archivos correspondientes a la versión HEAD-2



El funcionamiento es semejante al reset soft

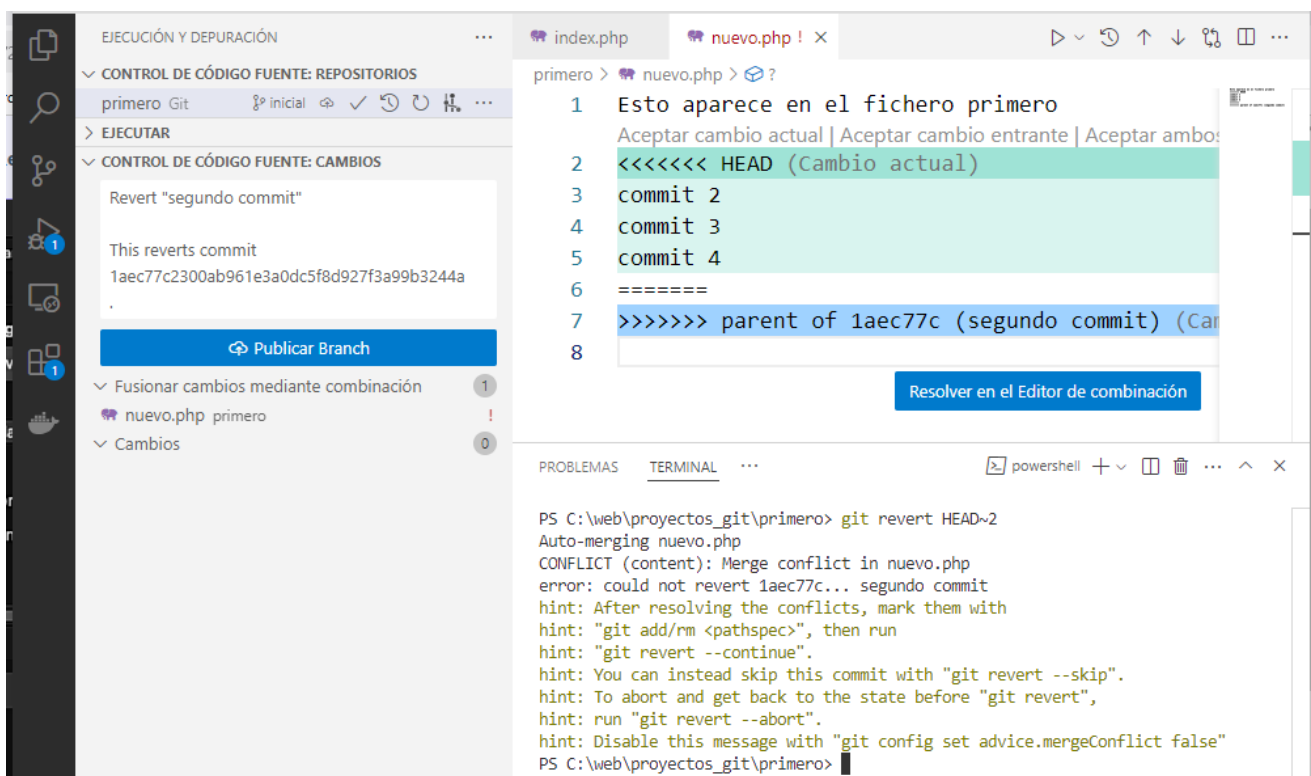
- **git reset --hard HEAD~2** Nos deja los archivos como estaban en la versión segunda.



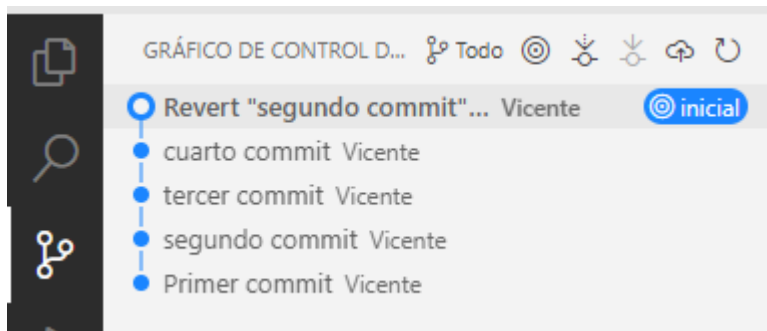
Borra todo rastro de las versiones posteriores.

Existe una forma de volver a versiones anteriores sin que el proceso sea destructivo: **git revert**.

Con este comando, se prepara un nuevo commit de vuelta al “segundo commit” donde nos deja el archivo para que lo editemos y escojamos que hacer con los cambios en versiones posteriores



Podemos editar la versión del archivo tal como queramos usando el Editor de Combinación y confirmar posteriormente para crear el commit.



El resultado lo podemos ver en el gráfico de control de versiones donde conviven las versiones previas con la nueva que hemos creado.

4.- Trabajo con ramas

Las ramas (branches) nos permiten desarrollar características nuevas o revisar bugs. En las ramas podemos tener nuestros propios commits y posteriormente realizar una mezcla de las mismas con otras.

Lo primero es crear una nueva rama mediante el comando **git branch nombre_rama**. Por ejemplo, podemos crear la rama segunda-rama con el comando **git branch segunda-rama**.

Después nos colocaremos en la rama mediante **git checkout segunda-rama**

```

PROBLEMAS  CONSOLA DE DEPURACIÓN  SALIDA  TERMINAL  PUERTOS

vicente@DESKTOP-G02SPOA MINGW64 /c/web/proyectos_git
• $ cd primero

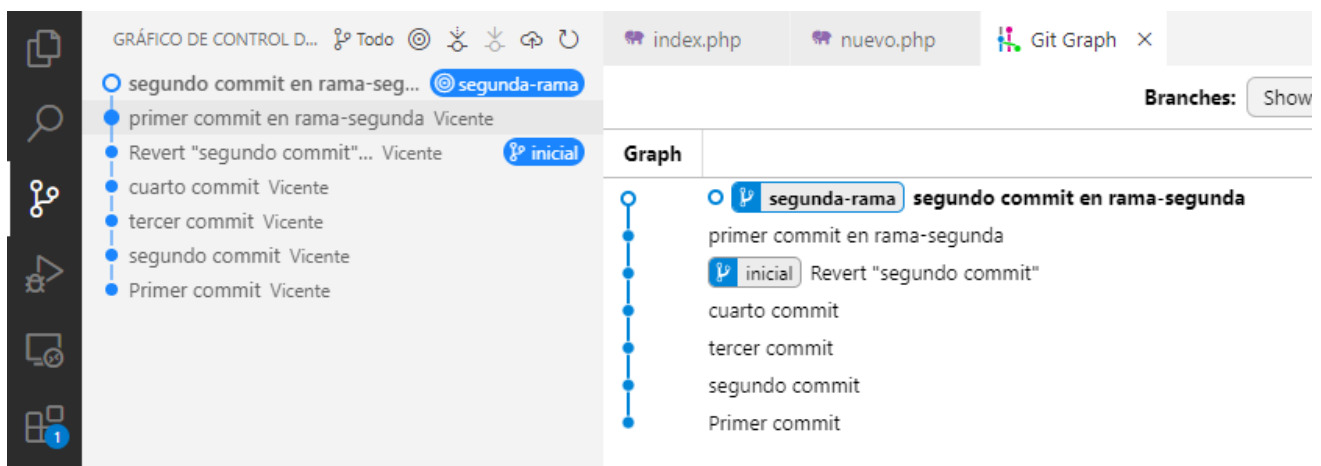
vicente@DESKTOP-G02SPOA MINGW64 /c/web/proyectos_git/primero (inicial)
• $ git checkout segunda-rama
Switched to branch 'segunda-rama'

vicente@DESKTOP-G02SPOA MINGW64 /c/web/proyectos_git/primero (segunda-rama)
• $

```

Las dos operaciones anteriores se pueden hacer con un solo comando: **git checkout -b segunda-rama**.

Si hacemos un commit tendremos un gráfico de versiones en el que podemos observar las dos ramas



[illegible]

The screenshot shows an IDE with a dark theme. On the left, the Git commit history is visible, showing a sequence of commits: 'Merge branch 'segunda-rama' into ini...' (initial), 'segundo commit en rama-seg...' (segunda-rama), 'primer commit en rama-segunda Vicente', and 'Revert "segundo commit"... Vicente'. The current commit is 'cuarto commit Vicente'. Below the history, a list of commits is shown: 'tercer commit Vicente', 'segundo commit Vicente', and 'Primer commit Vicente'.

The main editor area displays the content of 'nuevo.php', which contains three lines of code: '1 Esto aparece en el fichero primero', '2 commit 2', and '3 commit 3'. The file is named 'nuevo.php' and is in the 'primero' branch.

The terminal window at the bottom shows the following commands and output:

```
vicente@DESKTOP-G02SPOA MINGW64 /c/web/proyectos_git/primero (initial)
$ git checkout 697795142c1f707603668c42e214ad261c2c108a
Note: switching to '697795142c1f707603668c42e214ad261c2c108a'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you now to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 6977951 cuarto commit

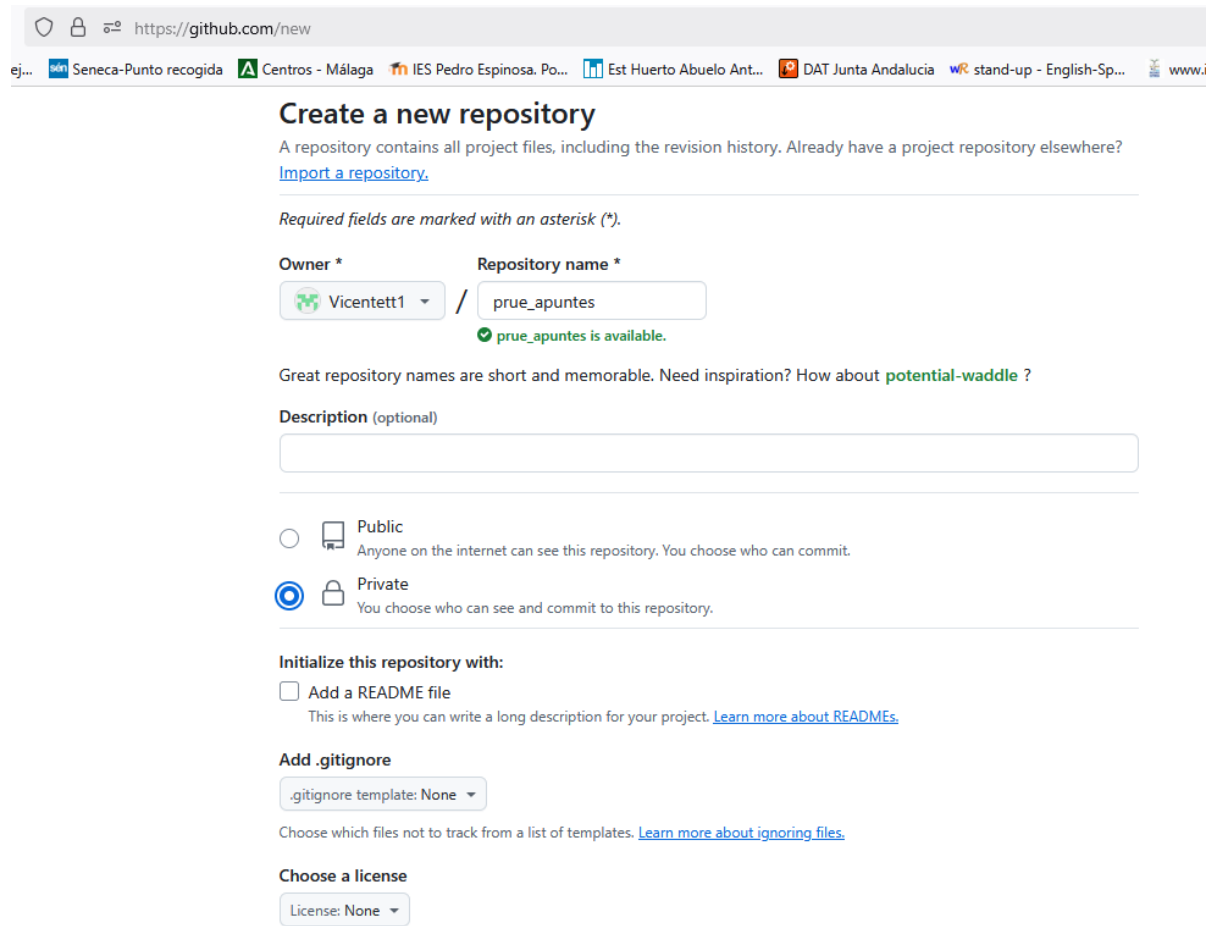
vicente@DESKTOP-G02SPOA MINGW64 /c/web/proyectos_git/primero ((6977951...))
$
```

5.- Trabajo con remotos

Visual Studio Code nos permite trabajar con servidores en la nube de git como GitHub y GitLab

En nuestro caso, vamos a trabajar con GitHub.

Creamos una cuenta en GitHub y creamos un repositorio nuevo.



https://github.com/new

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * / Repository name *

 Vicentett1 / prue_apuntes

 prue_apuntes is available.

Great repository names are short and memorable. Need inspiration? How about [potential-waddle](#) ?

Description (optional)

☐  Public
Anyone on the internet can see this repository. You choose who can commit.

☒  Private
You choose who can see and commit to this repository.

Initialize this repository with:

☐ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

 .gitignore template: None

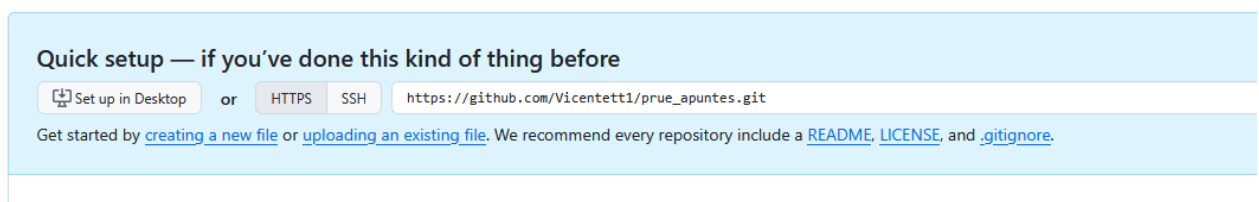
Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

 License: None

Definimos el nuevo repositorio como privado/publico, sin añadir archivo .gitignore ni readme, es decir vacío completamente.

Una vez creado tenemos que copiar la dirección del repositorio, en este caso https.



Quick setup — if you've done this kind of thing before

 Set up in Desktop or  HTTPS  SSH https://github.com/Vicentett1/prue_apuntes.git

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

Podemos usar la url SSH si hemos creado un certificado y registrado la clave pública en nuestra cuenta de Github

Nos vamos a Visual Studio y en el terminal añadimos el servidor remoto a Git con el comando **git remote add nombre url**

```
vicente@DESKTOP-G02SPOA MINGW64 /c/web/proyectos_git/primero (inicial)
$ git remote add github https://github.com/Vicentett1/prue_apuntes.git
```

A partir de aquí podemos volcar nuestro proyecto local a GitHub con el comando **git push remoto rama**

```
vicente@DESKTOP-G02SPOA MINGW64 /c/web/proyectos_git/primero (inicial)
● $ git push github inicial
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 8 threads
Compressing objects: 100% (21/21), done.
Writing objects: 100% (24/24), 2.17 KiB | 76.00 KiB/s, done.
Total 24 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), done.
To https://github.com/Vicentett1/prue_apuntes.git
 * [new branch]      inicial -> inicial
```

O traernos los cambios realizados en GitHub a nuestro proyecto con **git pull remoto rama**.

La primera vez que volquemos el proyecto a github tendremos que forzarlo usando **git push --set-upstream github inicial**. Es posible también que Visual Studio nos abra una ventana de GitHub de validación (usuario/contraseña si no tenemos iniciada sesión) y de asignación de permisos.

Podemos traernos también un proyecto de GitHub a una carpeta local vacía mediante **git clone url**

git data transport commands

patrickzahnd.ch

